

Prédiction du risque d'avalanche par apprentissage automatique

Nils Missillier

Candidat n°26257

Introduction



Introduction

Problématique

Comment estimer le risque d'avalanche à partir de données météorologiques historiques ?

SOYEZ DETECTABLE ! BE SEARCHABLE !

5 TRÈS FORT VERY HIGH



Conditions
très défavorables.

4 FORT HIGH



Conditions
défavorables. Forte instabilité
sur de nombreuses pentes.
L'appréciation du danger
requiert une grande expérience.

3 MARQUÉ CONSIDÉRABLE



Conditions
partiellement défavorables.
Instabilité marquée, parfois
sur de nombreuses pentes.
L'appréciation du danger
requiert de l'expérience.

2 LIMITÉ MODERATE



Conditions
favorables dans la plupart des cas.
Tenir compte des zones à risques
indiquées dans le bulletin
et suspectées sur le terrain.

1 FAIBLE LOW



Conditions
généralement favorables.

Source : Prefecture de Savoie

Introduction

Plan d'étude

- 1 **Gestion des données**
Extraction, corrélation, sélection
non-paramétrique
- 2 **Développement des modèles**
GLM (Poisson) et Random Forest
- 3 **Application**
Score de risque 1–5, comparaison



Source :Kirill Skorobogatko avalanche au sommet du
Khan Tengre



Gestion des données

Source des données : Météo-France

Variables météorologiques

- T : Température ($^{\circ}\text{C}$)
- P : Précipitations neigeuses (mm)
- H : Humidité relative (%)
- W : Vitesse du vent (km/h)
- ΔT : Gradient thermique
- ρ : Densité de la neige fraîche
- NF : Hauteur neige fraîche (cm)
- Y : Nombre d'avalanches (variable cible)

Structure matricielle

N observations, p variables :

$$X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,p} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,p} \end{pmatrix} \in \mathcal{M}_{N,p}(\mathbb{R})$$

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{N}^N$$

Analyse de corrélation

Matrice de corrélation

Soient V_i, V_j deux variables aléatoires :

$$\text{Cor}(V_i, V_j) = \frac{\text{Cov}(V_i, V_j)}{\sigma_{V_i} \sigma_{V_j}}$$

Pour X centrée réduite :

$$R = \frac{1}{N} X^T X \in \mathcal{M}_{p,p}(\mathbb{R})$$

R est **symétrique définie positive**, diagonalisable.

	Température maximale	Température minimale	Neige fraîche	Précipitations	Vitesse du vent
Température maximale	1,00	0,89	-0,54	-0,21	0,12
Température minimale	0,89	1,00	-0,48	-0,18	0,09
Neige fraîche	-0,54	-0,48	1,00	0,93	0,15
Précipitations	-0,21	-0,18	0,93	1,00	0,11
Vitesse du vent	0,12	0,09	0,15	0,11	1,00

Exemple de matrice de corrélation pour 5 variables

Sélection non-paramétrique des variables

Information Mutuelle (MI)

Critère **non-paramétrique** pour mesurer la dépendance entre une variable V_j et la cible Y :

$$I(V_j; Y) = \sum_y \sum_v p(v, y) \log \frac{p(v, y)}{p(v) p(y)}$$

Avantage : capture les non-linéarités, sans hypothèse distributionnelle.

Variables retenues

Après sélection sur données :

Dimension réduite :

$$p = 33 \rightarrow p^* = 9$$

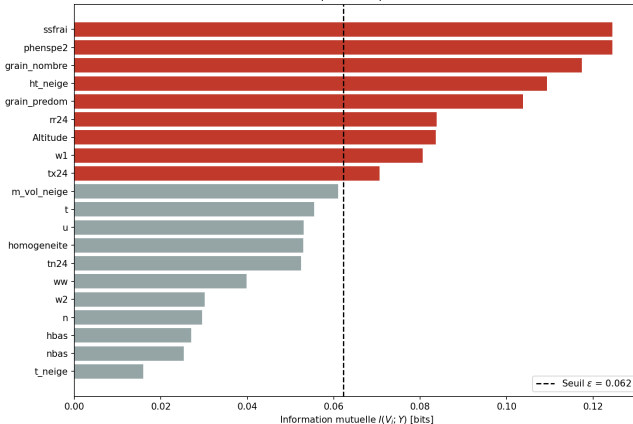
Critère de sélection

On retient les k variables maximisant $I(V_j; Y)$, avec un seuil ε :

$$\mathcal{S} = \{j \mid I(V_j; Y) > \varepsilon\}$$

Sélection non-paramétrique des variables

Selection non-parametrique des variables



Retenues ($I > \varepsilon$)

ssfrai Hauteur neige fraîche
phenspe2 Phénomène spécial n°2
grain_nombre Nb de grains de neige
ht_neige Hauteur de neige
grain_predom Type grain prédominant
rr24 Précipitations 24h
Altitude Altitude station
w1 Vitesse du vent
tx24 Temp. max. 24h

Rejetées ($I \leq \varepsilon$)

m_vol_neige Masse volumique neige
t Température
u Humidité relative
homogeneite Homogénéité manteau
tn24 Temp. min. 24h
ww Temps présent
w2 Vitesse du vent 2
n Nébulosité
hbas Hauteur base nuages
nbas Nébulosité basse
t_neige Température neige



Développement des modèles

GLM : Hypothèses et structure

Structure du Modèle Linéaire Généralisé

- **Distribution** : Le nombre d'avalanches Y suit une loi de Poisson de paramètre $\lambda > 0$.
- **Fonction de lien logarithmique** :

$$g(\lambda_i) = \ln(\lambda_i) = \mathbf{x}_i^\top \boldsymbol{\beta}$$

- **Modèle prédictif** :

$$\lambda_i = \exp\left(\sum_{k=1}^p \beta_k x_{ik}\right) = \exp(\mathbf{x}_i^\top \boldsymbol{\beta})$$

où $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)^\top$ est le vecteur des poids à optimiser.

Loi de Poisson

$$\mathbb{P}(Y = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Propriété clé :

$$\mathbb{E}[Y] = \text{Var}(Y) = \lambda$$

Famille exponentielle $\Rightarrow$
GLM valide

Optimisation : Log-Vraisemblance et Gradient

Log-Vraisemblance de Poisson

on maximise :

$$\mathcal{L}(\beta) = \sum_{i=1}^N \left(y_i (x_i^\top \beta) - \exp(x_i^\top \beta) - \ln(y_i!) \right)$$

Gradient (conditions du 1^{er} ordre)

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N x_{ij} (y_i - \lambda_i)$$

En notation vectorielle :

$$\nabla_{\beta} \mathcal{L} = X^\top (Y - \lambda) = 0$$

Matrice Hessienne

$$\frac{\partial^2 \mathcal{L}}{\partial \beta_j \partial \beta_k} = - \sum_{i=1}^N x_{ij} x_{ik} \lambda_i$$

$$H_{\mathcal{L}}(\beta) = -X^\top W X$$

avec $W = \text{diag}(\lambda_1, \dots, \lambda_N)$

H est **définie négative** $\Rightarrow \mathcal{L}$ est concave : maximum unique.

Newton-Raphson : Itérations des moindres carrés pondérés

Algorithme de Newton-Raphson

$$\beta^{(m+1)} = \beta^{(m)} - [H_{\mathcal{L}}(\beta^{(m)})]^{-1} \nabla_{\beta} \mathcal{L}(\beta^{(m)})$$

Réduction à un système linéaire

En posant le **pseudo-résidu ajusté** :

$$z_i = \sum_{k=1}^p x_{ik} \beta_k^{(m)} + \frac{y_i - \lambda_i}{\lambda_i}$$

on obtient l'équation **IRLS** :

$$\boxed{X^T W X \beta = X^T W z}$$

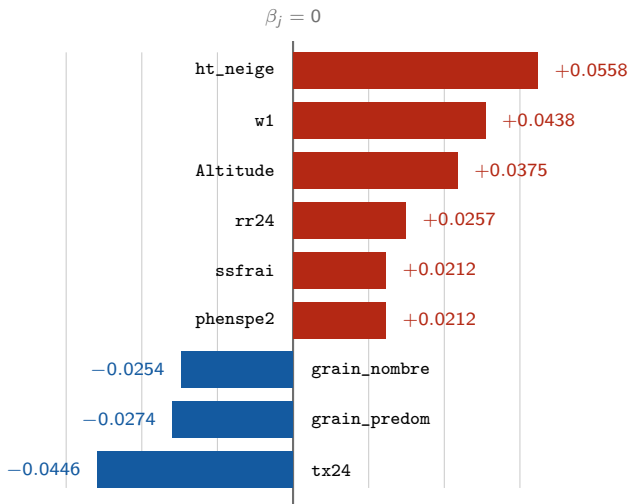
Résolu par **pivot de Gauss**.

Convergence garantie

$\mathcal{L}$ est strictement concave :

- Convergence **quadratique** locale
- Critère d'arrêt :
 $\|\beta^{(m+1)} - \beta^{(m)}\|_1 < \varepsilon$
- En pratique : < 20 itérations

GLM : coefficients β_j



Graphique des résultats des β_j après calcul

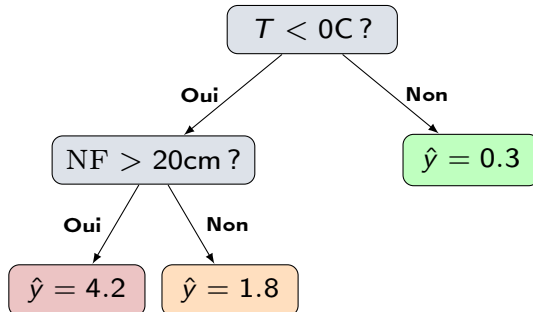
Random Forest : Construction d'un arbre

Critère de coupure : Réduction de Variance

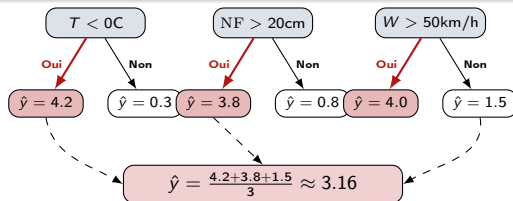
On cherche la variable j et le seuil s maximisant le gain :

$$\Delta_{\text{var}}(j, s) = \text{Var}(t) - \frac{N_{t_L}}{N_t} \text{Var}(t_L) - \frac{N_{t_R}}{N_t} \text{Var}(t_R)$$

avec $\text{Var}(t) = \frac{1}{N_t} \sum_{i \in t} (y_i - \bar{y}_t)^2$



Random Forest : Bagging et agrégation



Double randomisation

1. **Bootstrap (Bagging)** : chaque arbre h_b entraîné sur $\mathcal{D}_b^*$:

$$\mathcal{D}_b^* = \{(x_{i_k}, y_{i_k})\}_{k=1}^N, \quad i_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(\{1, \dots, N\})$$

2. **Sous-espace aléatoire** : seules $\lfloor \sqrt{p} \rfloor$ variables parmi p sont testées

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B h_b(x)$$

Comparaison des modèles

Critère	GLM (Poisson)	Random Forest
Nature	Paramétrique (Équation)	Non-paramétrique (Algo- rithmique)
Optimisation	Max. de Vraisemblance (Newton-Raphson)	Réduction de Variance (gain Δ_{var})
Effets de seuil (ex : $T < 0C$)	Lissés (faiblesse physique)	Capturés nativement
Interprétabilité	Excellente : poids β_j	Faible (boîte noire)
Données rares	Naturel (Poisson)	Nécessite équilibrage



Application des modèles

Résultats : Comparaison sur données test

Métriques sur 1761 jours test

Critère	GLM	RF
RMSE	0.610	0.580
MAE	0.473	0.440
Précision	58.9%	60.2%
Précision ± 1	98.9%	99.3%

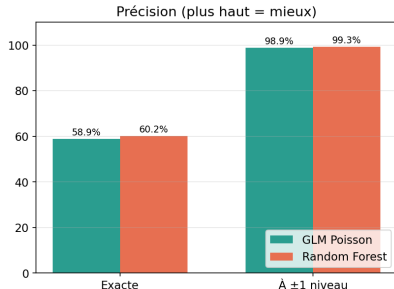
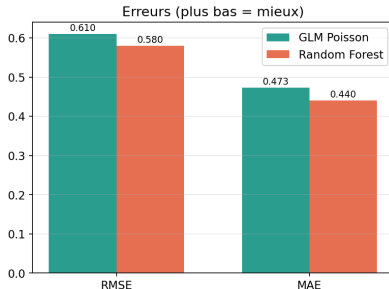
Définitions

RMSE (Root Mean Squared Error) :

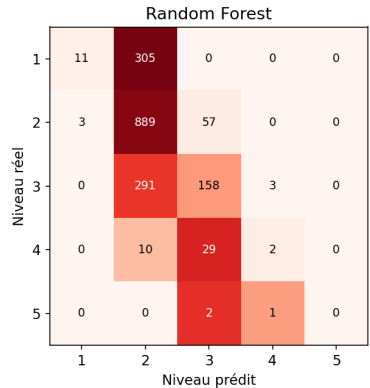
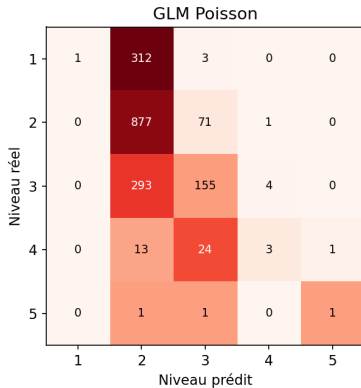
$$\sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

MAE (Mean Absolute Error) :

$$\frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$



Résultats : Matrices de confusion



Conclusion



Source : Des alpinistes sont bloqués par le passage d'une avalanche lors de leur ascension du Mont-Blanc, le 29 juillet 1994, à Chamonix (Haute-Savoie) - Pierre Bessard

Secourisme



Source : Des secours déployés dans la station à Tignes -

François Quivoron



Source : Avalanche rescue by organised rescue using avalanche transceiver, RECCO detector, rescue dog and probe line in parallel. :

Famille exponentielle et GLM

Définition : Famille exponentielle

Une loi appartient à la famille exponentielle si sa densité s'écrit :

$$f_X(x; \theta) = \exp(\eta(\theta) \cdot T(x) - A(\theta) + B(x))$$

Vérification pour la loi de Poisson

$$\mathbb{P}(Y_i = y_i) = \frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!} = \exp\left(\underbrace{y_i \ln \lambda_i - \lambda_i - \ln(y_i!)}_{\eta(\theta) \cdot T(x) - A(\theta) + B(x)}\right)$$

avec $\eta(\theta) = \ln \lambda_i = \mathbf{x}_i^\top \boldsymbol{\beta}$, soit la **fonction de lien canonique** :

$$g(\mathbb{E}[Y_i]) = \ln(\lambda_i) = \sum_{k=1}^p x_{ik} \beta_k$$

Initialisation des données

```
1 // --- INITIALISATION et LIBERATION ---
2
3 double **init_matrix(int n , int p ){
4     double** m = malloc( n* sizeof(double*));
5     for (int j =0 ;j<n; j++){
6         m[j] = calloc(p, sizeof(double)); /* calloc => initialise a 0 */
7     }
8     return m;
9
10 }
11
12 double * init_vect(int n){
13     double* v = malloc(n *sizeof(double));
14     for (int i = 0 ;i<n;i++){
15         v[i] = 0.0;
16     }
17     return v ;
18 }
19
20 void free_matrice (double** m, int n){
21     for (int i = 0 ;i<n;i++){
22         free(m[i]);
23     }
24     free(m);
25 }
```


Calcul matriciel

```
1
2 double** prod_matrcie (double** A , double** B, int n , int p, int m){
3     double** C = init_matrix (n,m);
4     for (int i = 0 ; i<n; i++){
5         for (int j =0 ; j<m; j++){
6             for (int k = 0; k<p;k++ ){
7                 C[i][j] += A[i][k] * B[k][j];
8             }
9         }
10    }
11    return C;
12 }
13
14 void prod_mat_vect (double**A,double*B, double* res ,int n , int p ){
15     for(int i = 0; i< n;i++){
16         res[i] = 0.0;
17         for (int j =0 ;j<p;j++){
18             res[i] += A[i][j] * B[j];
19         }
20     }
21 }
22
23 double** produit_XtW(double** X, double** W, int n, int p) {
24     double** R = init_matrix(p, n); // r sultat : p N
25     for (int j = 0; j < p; j++)
26         for (int k = 0; k < n; k++)
27             R[j][k] = X[k][j] * W[k][k]; // [X^T]_{jk} = X_{kj}, fois W_kk
28     return R;
29 }
```

Pivot de Gauss

```
1
2 void Pivot_de_gauss (double ** A , double * b , int n ){
3     double x [n];
4     for (int k =0 ; k<n-1;k++){
5         if (A[k][k] ==0 ){
6             printf ("impossible d'utiliser le pivot de Gauss");
7             exit (1);
8         }
9         for (int i = k+1 ;i<n;i++){
10            double coef = A[i][k] / A[k][k];
11            for(int j = k+1; j < n ; j++){
12                A[i][j] -= coef * A[k][j];
13            }
14            b[i] -= coef* b[k];
15            A[i][k] = 0;
16        }
17    }
18    x[n-1] = b[n-1] / A[n-1][n-1];
19    for (int i = n-2 ; i>=0;i--){
20        double somme = 0.0;
21        for (int j = i+1; j < n; j++){
22            somme += A[i][j] * x[j];
23        }
24        x[i] = (b[i] - somme) / A[i][i];
25    }
26
27    for (int i = 0; i<n; i++){
28        b[i] = x[i];
29    }
30 }
```

mise à jour des variables

```
1 void maj_eta (double* b, double* eta, double** X, int n , int p){
2     for(int i =0 ; i<n;i++){
3         eta[i] = 0.0;
4         for (int j=0 ; j<p;j++){
5             eta[i] += X[i][j]*b[j];
6         }
7     }
8 }
9
10 void maj_lambda(double* lambda, double* eta, int n ){
11     for(int i =0 ; i<n;i++){
12         lambda[i] = exp(eta[i]);
13     }
14 }
15
16 void maj_W(double* lambda, double** W, int n) {
17     for (int i = 0; i < n; i++) {
18         // On suppose W d j initialis e 0
19         W[i][i] = lambda[i];
20     }
21 }
22 void maj_z(double** X, double* Y, double* beta, double* lambda, double* z, int n, int p) {
23     for (int i = 0; i < n; i++) {
24         // Terme  $x_i^{-T} * beta = eta_i$  (on pourrait passer eta directement
25         // pour conomiser , mais on le recalcule ici pour lisibilit )
26         double eta_i = 0.0;
27         for (int k = 0; k < p; k++)
28             eta_i += X[i][k] * beta[k];
29         z[i] = eta_i + (Y[i] / lambda[i] - 1.0);
30     }
31 }
```

mise à jour des variables (suite)

```
1 void maj_b(double** X, double* Y, double* b, int n, int p, double tol) {
2     double* b_old = malloc(p * sizeof(double));
3     double* eta     = malloc(n * sizeof(double));
4     double* lambda   = malloc(n * sizeof(double));
5     double* z        = malloc(n * sizeof(double));
6     double** W       = init_matrix(n, n);
7     int iter = 0;
8     double diff = tol + 1.0;
9     const int MAX_ITER = 50;
10
11     while (diff > tol && iter < MAX_ITER) {
12         for (int i = 0; i < p; i++) {
13             b_old[i] = b[i];
14         }
15         maj_eta(b, eta, X, n, p);
16         maj_lambda(lambda, eta, n);
17         maj_W(lambda, W, n);
18         maj_z(X, Y, b, lambda, z, n, p);
19
20         double** XtW = produit_XtW(X, W, n, p);
21         double** A = prod_matrice(XtW, X, p, n, p); /* 5. Construire A = X^T W X (matrice p x p)
22             */
23         double* b_new = malloc(p * sizeof(double));
24         prod_mat_vect(XtW, z, b_new, p, n);
25         Pivot_de_gauss(A, b_new, p);
26         diff = 0.0;
27         for (int i = 0; i < p; i++) {
28             diff += fabs(b_new[i] - b_old[i]);
29             b[i] = b_new[i];
30         }
31         free_matrice(XtW, p);
32         free_matrice(A, p);
33         free(b_new);
34         iter++;
35     }
```

Standardiser

```
1 void standardiser(double** X, int n, int p, double* mean, double* std) {
2     /* On NE TOUCHE PAS la colonne 0 (intercept) */
3     mean[0] = 0.0;
4     std[0] = 1.0;
5     for (int j = 1; j < p; j++) {
6         /* Calcul de la moyenne */
7         double m = 0.0;
8         for (int i = 0; i < n; i++) m += X[i][j];
9         m /= n;
10        mean[j] = m;
11
12        /* Calcul de l'ecart-type */
13        double v = 0.0;
14        for (int i = 0; i < n; i++) {
15            double d = X[i][j] - m;
16            v += d * d;
17        }
18        v /= n;
19        double s = sqrt(v);
20        if (s < 1e-12) s = 1.0;    // protection si variance nulle
21        std[j] = s;
22
23        /* Application : X_centre = (X - m) / s */
24        for (int i = 0; i < n; i++) {
25            X[i][j] = (X[i][j] - m) / s;
26        }
27    }
28 }
29 void appliquer_standardisation(double** X, int n, int p, double* mean, double* std) {
30     for (int j = 1; j < p; j++) {
31         for (int i = 0; i < n; i++) {
32             X[i][j] = (X[i][j] - mean[j]) / std[j];
33         }
34     }
35 }
```

Prediction

```
1
2 void predire_et_evaluer(double** X, double* Y, int n, int p, double* b, const char* titre, const
   char* fichier_export) {
3     double* y_pred = malloc(n * sizeof(double));
4     /* Prediction :  $\lambda_i = \exp(x_i^T \beta)$  */
5     for (int i = 0; i < n; i++) {
6         double eta = 0.0;
7         for (int j = 0; j < p; j++) eta += X[i][j] * b[j];
8         y_pred[i] = exp(eta);
9     }
10    /* Metriques : RMSE et MAE */
11    double rmse = 0.0, mae = 0.0;
12    for (int i = 0; i < n; i++) {
13        double err = y_pred[i] - Y[i];
14        rmse += err * err;
15        mae += fabs(err);
16    }
17    rmse = sqrt(rmse / n);
18    mae = mae / n;
19
20    printf("\n==_EVALUATION_==\n", titre);
21    printf("_RMSE_=%.4f\n", rmse);
22    printf("_MAE_=%.4f\n", mae);
23
24    /* Export CSV pour comparaison ulterieure */
25    if (fichier_export != NULL) {
26        FILE* f = fopen(fichier_export, "w");
27        fprintf(f, "y_true,y_pred\n");
28        for (int i = 0; i < n; i++) {
29            fprintf(f, "%.0f,%.6f\n", Y[i], y_pred[i]);
30        }
31        fclose(f);
32        printf("_Predictions_exportees_dans_==\n", fichier_export);
33    }
34}
```

Compter lignes

```
1  int compter_lignes(const char* fichier) {
2      FILE* f = fopen(fichier, "r");
3      if (f == NULL) {
4          fprintf(stderr, "Impossible d'ouvrir_%s\n", fichier);
5          exit(1);
6      }
7      char buffer[1024];
8      fgets(buffer, sizeof(buffer), f); // on saute la ligne d'en-tete
9
10     int lignes = 0;
11     while (fgets(buffer, sizeof(buffer), f) != NULL) {
12         lignes++;
13     }
14     fclose(f);
15     return lignes;
16 }
```

Chargement des données

```
1 void charger_csv(const char* fichier, double*** X_out, double** Y_out, int* n_out, int p_features) {
2     /* 1. Compter les lignes pour allouer */
3     int n = compter_lignes(fichier);
4     int p = p_features + 1;          // +1 pour l'intercept
5
6     /* 2. Allouer X (n x p) et Y (n) */
7     double** X = init_matrix(n, p);
8     double* Y = malloc(n * sizeof(double));
9
10    /* 3. Ouvrir et sauter l'en-tete */
11    FILE* f = fopen(fichier, "r");
12    char buffer[1024];
13    fgets(buffer, sizeof(buffer), f);
14
15    /* 4. Lire ligne par ligne */
16    for (int i = 0; i < n; i++) {
17        X[i][0] = 1.0;                // INTERCEPT en colonne 0
18        fscanf(f, "%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf",
19            &X[i][1], &X[i][2], &X[i][3], &X[i][4], &X[i][5],
20            &X[i][6], &X[i][7], &X[i][8], &X[i][9], &Y[i]);
21    }
22    fclose(f);
23
24    printf("Charge %d lignes depuis %s\n", n, fichier);
25
26    *X_out = X;
27    *Y_out = Y;
28    *n_out = n;
29 }
```


main GLM

```
1  int main() {
2      const int P_FEATURES = 9;
3      const int p = P_FEATURES + 1;
4      double** X_train;
5      double* Y_train;
6      int      n_train;
7      charger_csv("train.csv", &X_train, &Y_train, &n_train, P_FEATURES);
8
9      double mean[10], std[10];
10     standardiser(X_train, n_train, p, mean, std);
11     printf("Standardisation effectuée.\n\n");
12     double* b = init_vect(p);
13     double y_mean = 0.0;
14     for (int i = 0; i < n_train; i++) y_mean += Y_train[i];
15     y_mean /= n_train;
16     b[0] = log(y_mean); // intercept = log(moyenne) : bon point de depart
17     printf("Initialisation: b[0] = log(mean Y) = %.4f\n\n", b[0]);
18
19     printf("---ENTRAINEMENT---\n");
20     maj_b(X_train, Y_train, b, n_train, p, 1e-6);
21
22     const char* noms[10] = {
23         "INTERCEPT", "ssfrac", "phenspe2", "grain_nombre", "ht_neige",
24         "grain_predom", "rr24", "Altitude", "w1", "tx24"
25     };
26     printf("\n---Coefficients et estimés---\n");
27     for (int j = 0; j < p; j++) {
28         printf("%13s = %.4f\n", noms[j], b[j]);
29     }
30
31     predire_et_evaluer(X_train, Y_train, n_train, p, b, "TRAIN", NULL);
32
33 }
```

main GLM (suite)

```
1
2
3     double** X_test;
4     double* Y_test;
5     int      n_test;
6     charger_csv("test.csv", &X_test, &Y_test, &n_test, P_FEATURES);
7     appliquer_standardisation(X_test, n_test, p, mean, std);
8
9     predire_et_evaluer(X_test, Y_test, n_test, p, b, "TEST", "predictions_glm.csv");
10
11     free_matrice(X_train, n_train);
12     free_matrice(X_test, n_test);
13     free(Y_train);
14     free(Y_test);
15     free(b);
16
17     return 0;
18 }
```

Structures Random Forest

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <math.h>
5  #include <time.h>
6  #include <float.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <sqlite3.h>
10
11 #define NUM_TREES 100      // Nombre d'arbres (r duit 10 pour le test rapide)
12 #define MAX_DEPTH 5       // Profondeur max
13 #define MIN_SAMPLES 3     // Minimum de donn es pour couper
14
15
16 // 1. Dataset : Contient la matrice X et le vecteur Y
17 typedef struct {
18     double **X;           // Matrice : X[ligne][colonne]
19     double *Y;            // Vecteur : Y[ligne]
20     int rows;             // Nombre d'observations (Jours)
21     int cols;             // Nombre de variables (M t o)
22 } Dataset;
23
24 // 2. N ud de l'arbre
25 typedef struct Node {
26     int is_leaf;          // 1 si feuille, 0 si n ud interne
27     double value;         // Pr diction (si feuille)
28
29     int feature_index;     // Indice de la variable coup e
30     double threshold;      // Seuil de coupure
31
32     struct Node *left;     // Enfants
33     struct Node *right;
34 } Node;
35
```

mise à jour des variables (suite)

```
1 // --- FONCTIONS UTILITAIRES & MEMOIRE ---
2
3 // Lib re proprement la m moire d'un Dataset
4 void Free_Dataset(Dataset *d) {
5     if (d == NULL) return;
6     for (int i = 0; i < d->rows; i++) {
7         free(d->X[i]); // Lib re chaque ligne
8     }
9     free(d->X); // Lib re le tableau de pointeurs
10    free(d->Y); // Lib re le vecteur cible
11    free(d);    // Lib re la structure
12 }
13
14 void Free_tree(Node* t){
15     if (t == NULL) return;
16     Free_tree(t->left);
17     Free_tree(t->right);
18     free(t);
19 }
20
21 void Free_forest(RandomForest* f ){
22     for (int i = 0 ; i<f->num_trees;i++){
23         Free_tree(f->trees[i]);
24     }
25     free(f->trees);
26     free(f);
27 }
```

Fonctions Maths

```
1 double Calcule_Variance(Dataset* data){
2     if (data->rows <= 1) return 0.0;
3
4     double somme = 0.0;
5     for (int i = 0; i < data->rows; i++){
6         somme += data->Y[i];
7     }
8     double moyenne = somme / data->rows;
9
10    double variance = 0.0;
11    for(int i = 0; i < data->rows; i++){
12        double ecart = data->Y[i] - moyenne;
13        variance += ecart * ecart;
14    }
15    return variance / data->rows;
16 }
17
18 double Calcule_Moyenne(Dataset* data) {
19     if (data->rows == 0) return 0.0;
20     double somme = 0.0;
21     for (int i = 0; i < data->rows; i++) somme += data->Y[i];
22     return somme / data->rows;
23 }
24
25 //tableau d'indice aleatoire entre 0 et sqrt(p)
26
27 int* random_tab (int p ){
28     int m = sqrt(p);
29     int* t = malloc(m* sizeof(int));
30     for (int i = 0 ; i<m;i++){
31         t[i] = rand() % p;
32     }
33     return t;
34 }
```

Donnée coupé en deux

```
1 void Split_Data(Dataset* parent, int feature_index, double seuil, Dataset** res_g, Dataset**  
   res_d){  
2  
3     //Compte le nombre de donn e en dessous du seuil  
4     int n_gauche = 0;  
5     int n_droit = 0;  
6     for (int i = 0; i < parent->rows; i++){  
7         if (parent->X[i][feature_index] < seuil) n_gauche++;  
8         else n_droit++;  
9     }  
10  
11     Dataset* gauche = malloc(sizeof(Dataset));  
12     Dataset* droit = malloc(sizeof(Dataset));  
13     gauche->rows = n_gauche; gauche->cols = parent->cols;  
14     droit->rows = n_droit;   droit->cols = parent->cols;  
15  
16     if (n_gauche > 0) {  
17         gauche->X = malloc(n_gauche * sizeof(double*));  
18         gauche->Y = malloc(n_gauche * sizeof(double));  
19         for(int i=0; i<n_gauche; i++) gauche->X[i] = malloc(gauche->cols * sizeof(double));  
20     } else { gauche->X = NULL; gauche->Y = NULL; }  
21  
22     if (n_droit > 0) {  
23         droit->X = malloc(n_droit * sizeof(double*));  
24         droit->Y = malloc(n_droit * sizeof(double));  
25         for(int i=0; i<n_droit; i++) droit->X[i] = malloc(droit->cols * sizeof(double));  
26     } else { droit->X = NULL; droit->Y = NULL; }
```

Donnée coupé en deux

```
1
2 //Copie les donn es
3 int idx_g = 0, idx_d = 0;
4 for (int i = 0; i < parent->rows; i++){
5     if (parent->X[i][feature_index] < seuil){
6         gauche->Y[idx_g] = parent->Y[i];
7         for(int c=0; c<parent->cols; c++) gauche->X[idx_g][c] = parent->X[i][c];
8         idx_g++;
9     } else {
10        droit->Y[idx_d] = parent->Y[i];
11        for(int c=0; c<parent->cols; c++) droit->X[idx_d][c] = parent->X[i][c];
12        idx_d++;
13    }
14 }
15 *res_g = gauche;
16 *res_d = droit;
17 }
```

Recherche de la meilleure coupure

```
1 // Cherche la meilleure question (Variable + Seuil)
2 BestSplit Find_Best_Split(Dataset* data) {
3     BestSplit best;
4     best.max_gain = -1.0;
5     best.feature_index = -1;
6     best.threshold = 0.0;
7
8     double current_var = Calcule_Variance(data);
9     if (current_var == 0 || data->rows < MIN_SAMPLES) return best;
10
11     int*indices = random_tab(data-> cols);
12
13     // On teste les variables tir es au sort
14     for (int r = 0; r < (int)sqrt(data->cols); r++) {
15
16         int feat = indices[r];
17
18         // On teste 10 seuils au hasard par variable (pour aller vite)
19         for (int attempt = 0; attempt < 10; attempt++) {
20             int random_idx = rand() % data->rows;
21             double seuil = data->X[random_idx][feat];
22
23             // Simulation rapide de Split_data sans malloc
24             int n_g = 0;
25             int n_d = 0;
26             double sum_g = 0;
27             double sum_d = 0;
28             double sq_sum_g = 0; // somme de acrr pour utiliser Koenig-Huygens
29             double sq_sum_d = 0;
```


Recherche de la meilleure coupure (suite)

```
1
2   for (int i = 0; i < data->rows; i++) {
3       double val = data->Y[i];
4       if (data->X[i][feat] < seuil) {
5           n_g++;
6           sum_g += val;
7           sq_sum_g += val*val;
8       } else {
9           n_d++;
10          sum_d += val;
11          sq_sum_d += val*val;
12      }
13  }
14  //si une des branches ne contient personne on peut passer a l'iteration suivante
15  if (n_g == 0 || n_d == 0) continue;
16  //calcule des variances
17  double var_l = (sq_sum_g/n_g) - (sum_g/n_g)*(sum_g/n_g);
18  double var_r = (sq_sum_d/n_d) - (sum_d/n_d)*(sum_d/n_d);
19  double weighted_var = ((double)n_g/data->rows)*var_l + ((double)n_d/data->rows)*var_r;
20
21  double gain = current_var - weighted_var;
22
23  if (gain > best.max_gain) {
24      best.max_gain = gain;
25      best.feature_index = feat;
26      best.threshold = seuil;
27  }
28  }
29  }
30  free(indices);
31  return best;
32  }
```

Tirage au sort des données

```
1 Dataset *Bootstrap_sample(Dataset*original){
2     Dataset* sample = malloc(sizeof(Dataset));
3     sample->rows = original->rows;
4     sample->cols= original->cols;
5     sample->X = malloc(sample->rows * sizeof(double));
6     sample-> Y = malloc(sample->rows * sizeof(double));
7
8     for (int i = 0 ; i<sample->rows;i++){
9         int r = rand() % original ->rows;
10        sample->X[i] = malloc( sample->cols* sizeof(double));
11        for (int j = 0 ; j<original -> cols; j++){
12            sample->X[i][j] = original->X[r][j];
13        }
14        sample->Y[i] = original ->Y[r];
15    }
16    return sample;
17 }
```

Construction de l'arbre

```
1 Node* Build_Tree(Dataset* data, int depth) {
2     Node* node = malloc(sizeof(Node));
3     //Conditions d'arr t
4     if (depth >= MAX_DEPTH || data->rows < MIN_SAMPLES) {
5         node->is_leaf = 1;
6         node->value = Calcule_Moyenne(data);
7         node->left = NULL; node->right = NULL;
8         return node;
9     }
10    BestSplit best = Find_Best_Split(data);
11
12    //Si on ne trouve pas de bon split (gain <= 0), on arr te
13    if (best.max_gain <= 0.000001) {
14        node->is_leaf = 1;
15        node->value = Calcule_Moyenne(data);
16        node->left = NULL;
17        node->right = NULL;
18        return node;
19    }
20    node->is_leaf = 0;
21    node->feature_index = best.feature_index;
22    node->threshold = best.threshold;
23    Dataset *gauche = NULL;
24    Dataset *droit = NULL;
25    Split_Data(data, best.feature_index, best.threshold, &gauche, &droit);
26
27    node->left = Build_Tree(gauche, depth + 1);
28    node->right = Build_Tree(droit, depth + 1);
29    Free_Dataset(gauche);
30    Free_Dataset(droit);
31    return node;
32 }
```

Prediction

```
1 double Predict_Tree(Node* node, double* input_features) {
2     if (node->is_leaf) {
3         return node->value;
4     }
5
6     if (input_features[node->feature_index] < node->threshold) {
7         return Predict_Tree(node->left, input_features);
8     } else {
9         return Predict_Tree(node->right, input_features);
10    }
11 }
12
13 // --- GESTION DE LA FORET ---
14
15 RandomForest* Train_Forest(Dataset* train_data, int n_trees) {
16     RandomForest* rf = malloc(sizeof(RandomForest));
17     rf->num_trees = n_trees;
18     rf->trees = malloc(n_trees * sizeof(Node*));
19
20     for(int i = 0; i < n_trees; i++) {
21         printf("Entrainement d'Arbre %d/%d...\n", i+1, n_trees);
22         Dataset* bootstrap = Bootstrap_sample(train_data); // dataset diff rent    chaque fois
23         rf->trees[i] = Build_Tree(bootstrap, 0);
24         Free_Dataset(bootstrap);
25     }
26     return rf;
27 }
```

Prediction (suite)

```
1 double Predict_Forest(RandomForest* rf, double* input) {  
2     double sum = 0.0;  
3     for(int i = 0; i < rf->num_trees; i++) {  
4         sum += Predict_Tree(rf->trees[i], input);  
5     }  
6     return sum / rf->num_trees;  
7 }
```

Chargement des données

```
1 Dataset* charger_csv(const char* fichier, int num_features) {
2     FILE* f = fopen(fichier, "r");
3     if (f == NULL) {
4         fprintf(stderr, "Impossible d'ouvrir %s\n", fichier);
5         exit(1);
6     }
7     char buffer[1024];
8
9     /* 1. Compter les lignes (en sautant l'en-tete) */
10    fgets(buffer, sizeof(buffer), f);
11    int rows = 0;
12    while (fgets(buffer, sizeof(buffer), f) != NULL) rows++;
13    rewind(f); /* on revient au debut du fichier */
14
15    /* 2. Allouer le Dataset */
16    Dataset* data = malloc(sizeof(Dataset));
17    data->rows = rows;
18    data->cols = num_features;
19    data->X = malloc(rows * sizeof(double));
20    data->Y = malloc(rows * sizeof(double));
21
22    /* 3. Lire les donnees (on re-saute l'en-tete) */
23    fgets(buffer, sizeof(buffer), f);
24    for (int i = 0; i < rows; i++) {
25        data->X[i] = malloc(num_features * sizeof(double));
26        fscanf(f, "%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf",
27            &data->X[i][0], &data->X[i][1], &data->X[i][2], &data->X[i][3],
28            &data->X[i][4], &data->X[i][5], &data->X[i][6], &data->X[i][7],
29            &data->X[i][8], &data->Y[i]);
30    }
31    fclose(f);
32    printf("Charge %d lignes depuis %s\n", rows, fichier);
33    return data;
34 }
```

main Random Forest

```
1  int main() {
2      printf("=== RANDOM FOREST - Risque d'avalanche ===\n\n");
3      srand(42); /* graine fixe : resultats reproductibles */
4
5      /* 1. Chargement TRAIN (9 features, pas d'intercept pour un arbre) */
6      Dataset* train = charger_csv("train.csv", 9);
7
8      /* 2. Entrainement de la foret */
9      printf("\n--- ENTRAINEMENT (%d arbres) ---\n", NUM_TREES);
10     RandomForest* rf = Train_Forest(train, NUM_TREES);
11
12     /* 3. Evaluation TRAIN */
13     evaluer(rf, train, "TRAIN", NULL);
14
15     /* 4. Chargement TEST + evaluation + export */
16     Dataset* test = charger_csv("test.csv", 9);
17     evaluer(rf, test, "TEST", "predictions_rf.csv");
18
19     /* 5. Liberation */
20     Free_Dataset(train);
21     Free_Dataset(test);
22     Free_forest(rf);
23
24     return 0;
25 }
```

Evaluation

```
1 void evaluer(RandomForest* rf, Dataset* data, const char* titre, const char* fout_name) {
2     double rmse = 0.0, mae = 0.0;
3     int ok = 0, ok1 = 0;
4
5     FILE* fout = NULL;
6     if (fout_name) {
7         fout = fopen(fout_name, "w");
8         fprintf(fout, "y_true,y_pred\n");
9     }
10
11     for (int i = 0; i < data->rows; i++) {
12         double p = Predict_Forest(rf, data->X[i]);
13         double e = p - data->Y[i];
14         rmse += e * e;
15         mae += fabs(e);
16
17         /* accuracy : on arrondit la prediction au niveau le plus proche */
18         int pa = (int)(p + 0.5);
19         if (pa < 1) pa = 1;
20         if (pa > 5) pa = 5;
21         int diff = abs(pa - (int)data->Y[i]);
22         if (diff == 0) ok++;
23         if (diff <= 1) ok1++;
24
25         if (fout) fprintf(fout, "%.0f,%.6f\n", data->Y[i], p);
26     }
27     rmse = sqrt(rmse / data->rows);
28     mae = mae / data->rows;
29
30     printf("\n==_EVALUATION_==\n", titre);
31     printf("_RMSE_=%.4f\n", rmse);
32     printf("_MAE_=%.4f\n", mae);
33     printf("_Accuracy_exacte_=%.1f%%\n", 100.0 * ok / data->rows);
34     printf("_Accuracy_u+1_=%.1f%%\n", 100.0 * ok1 / data->rows);
35     if (fout) {
```


Méthode 3x3 de Werner Munter

Analyse du risque	Les conditions	Le terrain	L'humain
Avant la sortie	BERA Prévisions météo Réseaux sociaux Situations typiquement dangereuses	Cartes, topos, Horaires prévus Inclinaisons, formes et orientations des pentes Scénarios à éviter Yéti, Skitourenguru	Nombre de participants Expérience et équipement Attentes
Pendant l'approche	Météo locale QCNF Signaux d'alarmes	Fréquentation Comparaison carte/terrain	Forme du jour Contrôle DVA Tenue de l'horaire
Dans les passages clefs	Météo, visibilité Réchauffement ou humidification de la neige QCNF Signaux d'alarme	Qualité de la trace, configuration du terrain Inclinaison et orientation réelles Autres dangers objectifs	Forme du jour Tenue de l'horaire Respect des distances de sécurité Corridor de descente

Source : Montagne Magazine

Matrice de corrélation

Hypothèse. Les p colonnes de X sont **centrées réduites** :

$$\forall j, \quad \frac{1}{N} \sum_{i=1}^N x_{ij} = 0 \quad \text{et} \quad \frac{1}{N} \sum_{i=1}^N x_{ij}^2 = 1$$

Covariance empirique. Pour V_j, V_k centrées réduites ($\sigma_j = \sigma_k = 1$) :

$$\text{Cov}(V_j, V_k) = \frac{1}{N} \sum_{i=1}^N x_{ij} x_{ik} = \text{Cor}(V_j, V_k)$$

Lien avec $X^\top X$. L'élément (j, k) de $X^\top X$ est :

$$[X^\top X]_{jk} = \sum_{i=1}^N [X^\top]_{ji} X_{ik} = \sum_{i=1}^N x_{ij} x_{ik} = N \cdot \text{Cor}(V_j, V_k)$$

$$\Rightarrow \quad R = \frac{1}{N} X^\top X \in \mathcal{M}_{p,p}(\mathbb{R})$$

Matrice de corrélation — caractère SDP

Symétrie.

$$R^\top = \frac{1}{N}(X^\top X)^\top = \frac{1}{N} X^\top (X^\top)^\top = \frac{1}{N} X^\top X = R$$

Semi-définie positive. Pour tout $u \in \mathbb{R}^p$:

$$u^\top R u = \frac{1}{N} u^\top X^\top X u = \frac{1}{N} (Xu)^\top (Xu) = \frac{1}{N} \|Xu\|_2^2 \geq 0$$
$$\implies \boxed{R \succeq 0}$$

R diagonalisable en base orthonormée (théorème spectral).

Définie positive.

$$u^\top R u = 0 \iff \|Xu\|^2 = 0 \iff Xu = 0 \iff u \in \ker X$$

$$\boxed{R \succ 0 \iff \ker X = \{0\} \iff X \text{ de rang plein } p}$$

Information mutuelle

Définition.

$$I(V; Y) = \sum_v \sum_y p(v, y) \log \frac{p(v, y)}{p(v) p(y)}$$

Reformulation $I(V; Y) = H(Y) - H(Y | V)$. En développant avec $p(v, y) = p(v) p(y|v)$:

$$\begin{aligned} I(V; Y) &= \sum_{v,y} p(v, y) \log p(y|v) - \sum_{v,y} p(v, y) \log p(y) \\ &= \sum_v p(v) \sum_y p(y|v) \log p(y|v) - \sum_y p(y) \log p(y) \\ &= -H(Y | V) + H(Y) \\ &\Rightarrow \boxed{I(V; Y) = H(Y) - H(Y | V)} \end{aligned}$$

Information mutuelle

Positivité. On écrit :

$$-I(V; Y) = \sum_{v,y} p(v,y) \log \frac{p(v)p(y)}{p(v,y)} = \mathbb{E}_{p(v,y)} \left[\log \frac{p(v)p(y)}{p(v,y)} \right]$$

$\log$ est **concave** $\Rightarrow$ inégalité de Jensen :

$$-I \leq \log \mathbb{E}_{p(v,y)} \left[\frac{p(v)p(y)}{p(v,y)} \right]$$

On calcule l'espérance :

$$\mathbb{E}_{p(v,y)} \left[\frac{p(v)p(y)}{p(v,y)} \right] = \sum_{v,y} p(v,y) \cdot \frac{p(v)p(y)}{p(v,y)} = \sum_{v,y} p(v)p(y) = 1$$

Donc $-I \leq \log 1 = 0$, soit $I(V; Y) \geq 0$.

Information mutuelle 2

Cas d'égalité. $\log$ *strictement* concave : égalité dans Jensen $\Leftrightarrow$ la v.a. $\frac{p(v)p(y)}{p(v,y)}$ est p.s. constante = 1, soit :

$$I(V; Y) = 0 \iff p(v, y) = p(v)p(y) \forall (v, y) \iff V \perp\!\!\!\perp Y$$

GLM Poisson — construction de la log-vraisemblance

Modèle. $Y_i \sim \mathcal{P}(\lambda_i)$ indépendants, $\lambda_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta}) > 0$.

Vraisemblance. Par indépendance :

$$L(\boldsymbol{\beta}) = \prod_{i=1}^N \mathbb{P}(Y_i = y_i) = \prod_{i=1}^N \frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!}$$

Log-vraisemblance. log strictement croissant (même argmax) :

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^N \log\left(\frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!}\right) = \sum_{i=1}^N [y_i \log \lambda_i - \lambda_i - \log(y_i!)]$$

En substituant $\log \lambda_i = \mathbf{x}_i^\top \boldsymbol{\beta}$ et $\lambda_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta})$:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^N \left[y_i (\mathbf{x}_i^\top \boldsymbol{\beta}) - \exp(\mathbf{x}_i^\top \boldsymbol{\beta}) - \ln(y_i!) \right]$$

GLM Poisson — calcul du gradient

Dérivation en chaîne. Posons $\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$. On a :

$$\frac{\partial \eta_i}{\partial \beta_j} = x_{ij}, \quad \lambda_i = e^{\eta_i}, \quad \frac{\partial \lambda_i}{\partial \eta_i} = e^{\eta_i} = \lambda_i$$

Pour chaque terme $\mathcal{L}_i = y_i \eta_i - e^{\eta_i} - \ln(y_i!)$:

$$\frac{\partial \mathcal{L}_i}{\partial \eta_i} = y_i - e^{\eta_i} = y_i - \lambda_i$$

Par la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}_i}{\partial \eta_i} \cdot \frac{\partial \eta_i}{\partial \beta_j} = \sum_{i=1}^N x_{ij} (y_i - \lambda_i)$$

En notation vectorielle (j -ième composante de $\mathbf{X}^\top (\mathbf{Y} - \boldsymbol{\lambda})$) :

$$\nabla_{\boldsymbol{\beta}} \mathcal{L} = \mathbf{X}^\top (\mathbf{Y} - \boldsymbol{\lambda})$$

Hessienne — calcul de $H_{\mathcal{L}}(\beta)$

Point de départ. On dérive $\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N x_{ij}(y_i - \lambda_i)$ par rapport à β_k .

Dérivée de λ_i par rapport à β_k .

$$\frac{\partial \lambda_i}{\partial \beta_k} = \frac{\partial}{\partial \beta_k} \exp(x_i^\top \beta) = x_{ik} \exp(x_i^\top \beta) = x_{ik} \lambda_i$$

Dérivée croisée.

$$\frac{\partial^2 \mathcal{L}}{\partial \beta_j \partial \beta_k} = \sum_{i=1}^N x_{ij} \frac{\partial (y_i - \lambda_i)}{\partial \beta_k} = - \sum_{i=1}^N x_{ij} x_{ik} \lambda_i$$

Forme matricielle. En posant $W = \text{diag}(\lambda_1, \dots, \lambda_N)$:

$$[X^\top W X]_{jk} = \sum_{i=1}^N x_{ij} \lambda_i x_{ik} \implies \boxed{H_{\mathcal{L}}(\beta) = -X^\top W X}$$

Hessienne — caractère défini négatif

Forme quadratique. Pour tout $u \in \mathbb{R}^p$:

$$\begin{aligned} u^\top H_{\mathcal{L}} u &= -u^\top X^\top W X u = -(Xu)^\top W (Xu) \\ &= -\sum_{i=1}^N W_{ii} (Xu)_i^2 \\ &= -\sum_{i=1}^N \lambda_i (Xu)_i^2 \leq 0 \end{aligned}$$

car $\lambda_i = e^{\eta_i} > 0$ et $(Xu)_i^2 \geq 0$.

Égalité ssi $\lambda_i (Xu)_i^2 = 0$ pour tout i , i.e. $Xu = 0$.

Si X est de rang plein : $Xu = 0 \Rightarrow u = 0$, donc :

$H_{\mathcal{L}}(\beta) \prec 0 \iff \mathcal{L} \text{ strictement concave sur } \mathbb{R}^p$

Maximum global de $\mathcal{L}$ **unique**.

Newton-Raphson — mise en forme de l'itération

Itération de Newton.

$$\beta^{(m+1)} = \beta^{(m)} - [H_{\mathcal{L}}(\beta^{(m)})]^{-1} \nabla_{\beta} \mathcal{L}(\beta^{(m)})$$

En substituant $H_{\mathcal{L}} = -X^{\top} W X$ et $\nabla_{\beta} \mathcal{L} = X^{\top} (Y - \lambda)$:

$$\beta^{(m+1)} = \beta^{(m)} + (X^{\top} W X)^{-1} X^{\top} (Y - \lambda)$$

Multiplication à gauche par $X^{\top} W X$.

$$X^{\top} W X \beta^{(m+1)} = X^{\top} W X \beta^{(m)} + X^{\top} (Y - \lambda)$$

Newton-Raphson — réduction à l'IRLS

Réécriture du membre de droite. Comme

$$X^T(Y - \lambda) = X^T W [W^{-1}(Y - \lambda)] :$$

$$X^T W X \beta^{(m+1)} = X^T W \underbrace{[X \beta^{(m)} + W^{-1}(Y - \lambda)]}_z$$

Pseudo-résidu ajusté.

$$z_i = (x_i^T \beta^{(m)}) + \frac{y_i - \lambda_i}{\lambda_i} \quad \Longleftrightarrow \quad z = X \beta^{(m)} + W^{-1}(Y - \lambda)$$

D'où le système **IRLS** (Iteratively Reweighted Least Squares) :

$$\boxed{X^T W X \beta^{(m+1)} = X^T W z}$$

C'est l'équation normale de **moindres carrés pondérés** :

$$\beta^{(m+1)} = \arg \min_{\beta} \sum_{i=1}^N \lambda_i (z_i - x_i^T \beta)^2$$

Résolu à chaque itération par **pivot de Gauss**.

Pivot de Gauss — pivots strictement positifs

Système. $A\beta = b$ avec $A = X^\top W X$ **symétrique définie positive**.

Critère de Sylvester. $A \succ 0 \iff$ tous les mineurs principaux dominants sont > 0 :

$$\Delta_k = \det(A_k) > 0, \quad k = 1, \dots, p$$

où A_k est le sous-mineur principal d'ordre k .

Pivots de l'élimination. Le k -ième pivot est :

$$\pi_k = \frac{\Delta_k}{\Delta_{k-1}}$$

Comme $\Delta_k > 0$ et $\Delta_{k-1} > 0$:

$$\pi_k = \frac{\Delta_k}{\Delta_{k-1}} > 0 \quad \forall k = 1, \dots, p$$

Le pivot ne s'annule jamais — l'élimination est numériquement bien définie.

Pivot de Gauss — algorithme et complexité

Phase 1 — Élimination (triangularisation $A \rightarrow U$). Pour $k = 1, \dots, p - 1$:

- Pivot partiel : échange de la ligne k avec $\arg \max_{i \geq k} |A_{ik}|$ (stabilité).
- Pour $i = k + 1, \dots, p$: $m_{ik} = A_{ik}/A_{kk}$, $A_{i,\cdot} \leftarrow A_{i,\cdot} - m_{ik} A_{k,\cdot}$,
 $b_i \leftarrow b_i - m_{ik} b_k$.

Phase 2 — Remontée ($U\beta = b'$). Pour $i = p, p - 1, \dots, 1$:

$$\beta_i = \frac{1}{U_{ii}} \left(b'_i - \sum_{j=i+1}^p U_{ij} \beta_j \right)$$

Complexité totale.

$\mathcal{O}(p^3)$ opérations (élimination) + $\mathcal{O}(p^2)$ (remontée)

Random Forest — prédiction optimale d'une feuille

Problème. Dans une feuille t (ensemble d'indices), on cherche la constante c minimisant l'erreur quadratique :

$$\min_{c \in \mathbb{R}} \sum_{i \in t} (y_i - c)^2$$

Condition du premier ordre.

$$\frac{d}{dc} \sum_{i \in t} (y_i - c)^2 = -2 \sum_{i \in t} (y_i - c) = 0 \implies \sum_{i \in t} y_i = N_t c \implies \boxed{c^* = \bar{y}_t}$$

Condition du second ordre. $\frac{d^2}{dc^2} \sum (y_i - c)^2 = 2N_t > 0$: c'est bien un minimum.

Mesure d'hétérogénéité de la feuille t .

$$\text{Var}(t) = \frac{1}{N_t} \sum_{i \in t} (y_i - \bar{y}_t)^2 = \frac{1}{N_t} \sum_{i \in t} y_i^2 - \bar{y}_t^2$$

Random Forest — gain d'une coupure

Coupure (j, s) . On partitionne t en :

$$t_L = \{i \in t : x_{ij} < s\}, \quad t_R = \{i \in t : x_{ij} \geq s\}, \quad N_t = N_{t_L} + N_{t_R}$$

Gain de la coupure.

$$\Delta_{\text{var}}(j, s) = \text{Var}(t) - \frac{N_{t_L}}{N_t} \text{Var}(t_L) - \frac{N_{t_R}}{N_t} \text{Var}(t_R)$$

Équivalence. $\text{Var}(t)$ est **fixé** pour le nœud courant :

$$\begin{aligned} \arg \max_{(j,s)} \Delta_{\text{var}} &= \arg \max_{(j,s)} \left[-\frac{N_{t_L}}{N_t} \text{Var}(t_L) - \frac{N_{t_R}}{N_t} \text{Var}(t_R) \right] \\ &= \arg \min_{(j,s)} \left[\frac{N_{t_L}}{N_t} \text{Var}(t_L) + \frac{N_{t_R}}{N_t} \text{Var}(t_R) \right] \end{aligned}$$

$$(j^*, s^*) = \arg \min_{(j,s)} \left[\frac{N_{t_L}}{N_t} \text{Var}(t_L) + \frac{N_{t_R}}{N_t} \text{Var}(t_R) \right]$$

Variance RF — développement par bilinéarité

Cadre. $h_1, \dots, h_B$ arbres i.d. avec :

$$\text{Var}(h_b) = \sigma^2, \quad \text{Cov}(h_b, h_{b'}) = \begin{cases} \sigma^2 & b = b' \\ \rho\sigma^2 & b \neq b' \end{cases}$$

Prédicteur agrégé : $\hat{y}_{\text{RF}} = \frac{1}{B} \sum_{b=1}^B h_b$. **Calcul.** Par bilinéarité de la covariance :

$$\begin{aligned} \text{Var}(\hat{y}_{\text{RF}}) &= \frac{1}{B^2} \sum_{b=1}^B \sum_{b'=1}^B \text{Cov}(h_b, h_{b'}) = \frac{1}{B^2} \left[\underbrace{\sum_{b=b'} \sigma^2}_{B \text{ termes}} + \underbrace{\sum_{b \neq b'} \rho\sigma^2}_{B(B-1) \text{ termes}} \right] \\ &= \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] \end{aligned}$$

Variance RF — simplification et plancher

Simplification.

$$\begin{aligned}\text{Var}(\hat{y}_{\text{RF}}) &= \frac{B\sigma^2 + B(B-1)\rho\sigma^2}{B^2} = \frac{\sigma^2}{B} + \frac{(B-1)\rho\sigma^2}{B} \\ &= \rho\sigma^2 + \frac{\sigma^2 - \rho\sigma^2}{B}\end{aligned}$$

$$\boxed{\text{Var}(\hat{y}_{\text{RF}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2}$$

Plancher de corrélation. Quand $B \rightarrow \infty$:

$$\frac{1-\rho}{B}\sigma^2 \rightarrow 0 \quad \implies \quad \text{Var}(\hat{y}_{\text{RF}}) \rightarrow \rho\sigma^2$$

$$\boxed{\inf_{B \geq 1} \text{Var}(\hat{y}_{\text{RF}}) = \rho\sigma^2}$$

Hypothèse de Poisson. Le modèle impose l'**équidispersion** :

$$\mathbb{E}[Y_i] = \text{Var}(Y_i) = \lambda_i.$$

Si la variance réelle dépasse la moyenne, l'hypothèse est violée et les écarts-types des $\hat{\beta}_j$ sont sous-estimés.

Statistique de dispersion de Pearson. On confronte les résidus à la variance théorique λ_i :

$$\hat{\varphi} = \frac{1}{N-p} \sum_{i=1}^N \frac{(y_i - \hat{\lambda}_i)^2}{\hat{\lambda}_i}, \quad \hat{\lambda}_i = \exp(\mathbf{x}_i^\top \hat{\boldsymbol{\beta}}).$$

Surdispersion 2

Règle de décision.

$$\hat{\varphi} \approx 1 \implies \text{Poisson valide}, \quad \hat{\varphi} \gg 1 \implies \boxed{\text{surdispersion}}.$$

Sur mes données ($N = \langle 1761 \rangle$, $p = 10$) :

$$\hat{\varphi} = \langle 2, 3 \rangle.$$

Parade : la binomiale négative. On rend λ aléatoire ($\lambda \sim \text{Gamma}$, paramètre de dispersion α) :

$$\text{Var}(Y) = \mu + \alpha\mu^2 > \mu.$$

Surdispersion et binomiale négative

Hypothèse Poisson. La loi $\mathcal{P}(\lambda)$ impose l'**équidispersion** :

$$\mathbb{E}[Y] = \text{Var}(Y) = \lambda.$$

Statistique de dispersion. On estime :

$$\hat{\phi} = \frac{1}{N - p} \sum_{i=1}^N \frac{(y_i - \hat{\lambda}_i)^2}{\hat{\lambda}_i}$$

$\hat{\phi} \approx 1$: Poisson valide. $\hat{\phi} \gg 1$: **surdispersion**.

Binomiale négative (mélange Poisson-Gamma). On pose $\lambda \sim \text{Gamma}(\mu/\alpha, \alpha)$, $Y \mid \lambda \sim \mathcal{P}(\lambda)$. Par la loi des variances totales :

$$\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}[Y \mid \lambda]] = \mathbb{E}[\lambda] = \mu$$

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y \mid \lambda)] + \text{Var}(\mathbb{E}[Y \mid \lambda]) = \mathbb{E}[\lambda] + \text{Var}(\lambda)$$

Pour une loi Gamma de paramètre α : $\text{Var}(\lambda) = \alpha\mu^2$, d'où :

$$\text{Var}(Y) = \mu + \alpha\mu^2 > \mu$$

Métriques de validation

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Précision exacte : $\hat{y}_i = y_i$. **Précision ± 1 :** $|\hat{y}_i - y_i| \leq 1$.

Inégalité entre les métriques. Par l'inégalité de Cauchy-Schwarz :

$$\text{MAE} = \frac{1}{n} \sum_i |\hat{y}_i - y_i| \cdot 1 \leq \sqrt{\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2} \cdot \sqrt{\frac{1}{n} \sum_i 1^2} = \text{RMSE}$$

$$\boxed{\text{MAE} \leq \text{RMSE}}$$

Biais par fuite temporelle. Un découpage aléatoire introduit des paires (x_i, x_j) corrélées entre train et test. Le score de test estime alors :

$$\mathbb{E}_{\text{alatoire}}[\text{err}] \leq \mathbb{E}_{\text{temporel}}[\text{err}]$$

La validation temporelle donne une borne supérieure plus honnête.